TreeCast: Multi-Party Key Establishment Protocol for IoT Devices

Supriyo Banerjee
Indian Institute of Technology Madras
Chennai, India
supriyobanerjee537@gmail.com

Sayon Duttagupta
COSIC, KU Leuven
Leuven, Belgium
Sayon.Duttagupta@esat.kuleuven.be

***Abstract*—Secure communication in the Internet of Things (IoT) requires lightweight protocols that scale across unicast, multicast, and broadcast settings. Existing solutions typically depend on centralized gateways, which introduce single points of failure and scalability limitations. We present *TreeCast*, a decentralized group key establishment protocol that organizes devices in a binary tree and derives communication keys through hybrid key exchange. The protocol achieves efficient and scalable key management, supporting dynamic membership with localized rekeying. We provide a formal security analysis and proof, showing that *TreeCast* achieves authentication, session key confidentiality, post-compromise security, and partial forward secrecy. In addition, we evaluate computational and storage costs of the protocol, demonstrating logarithmic scalability in both communication overhead and device state. By enabling a single framework for unicast, multicast, and broadcast communication, our approach bridges the gap between cryptographic rigor and practical IoT deployment. *TreeCast* provides a deployable, communication-oriented solution to secure large-scale IoT networks.**

***Index Terms*—Tree-based Group Key Establishment, IoT Security, Decentralized Key Establishment, Multicast, Broadcast**

I. Introduction

The number of Internet of Things (IoT) devices continues to grow rapidly [1]. These devices increasingly communicate autonomously, without constant human supervision, to coordinate sensing and actuation in consumer and industrial environments. Within such domains, a single device often needs to deliver information or commands either to one peer, to a subset of peers, or to all devices. Securing these communications is central to safety, privacy, and resilience.

We focus on three communication modes that arise naturally in device-to-device operation. In *unicast*, a device sends to exactly one peer. In *multicast*, a device sends to a selected subset. In *broadcast*, a device sends to all devices in the domain. These modes are common in practice. For example, a smart water meter may send usage data only to the homeowner's phone (*unicast*), share water quality with all water-using appliances (*multicast*), or broadcast a leakage alert to every device. In industrial control systems, a programmable logic controller may multicast a recipe to actuators, broadcast an emergency stop to the line, or unicast a diagnostic request to one sensor. Failures in these scenarios can cause unsafe actuation, downtime, or privacy violations, making secure communication essential.

Current group key establishment (GKE) protocols are poorly aligned with these needs. Standards such as Matter [2] rely on a central hub or gateway, which simplifies onboarding but creates a single point of failure, performance bottlenecks, and difficulties when devices are intermittently offline. Human-in-the-loop methods based on barcodes, shaking, or gestures [3], [4], [5] are useful for very small groups but do not scale. Symmetric approaches with a Key Distribution Center (KDC) [6] are efficient but every join or leave event forces redistribution of a fresh group key, which becomes costly at scale. Pure public key schemes avoid these limitations but often impose heavy computation or state that low power devices cannot sustain.

These challenges highlight the need for a mechanism that is decentralized in normal operation, supports the three communication modes, admits efficient addition and revocation through localized updates, and remains lightweight for constrained devices. The protocol must provide authentication, confidentiality, key freshness, forward secrecy, and recovery after compromise, while scaling predictably with deployment size. In this paper, we want to solve this open problem and propose *TreeCast*, a novel key-establishment protocol designed for such communication use cases in the IoT domain. Our work addresses these needs by organizing devices as leaves of a binary tree and deriving communication keys along that structure: pairwise keys for *unicast*, least common ancestor keys for *multicast*, and the root key for *broadcast*. Public key operations derive short-lived session secrets, with symmetric primitives securing data. Membership changes trigger updates only along a path in the tree, bounding rekeying cost and supporting asynchronous operation. The *TreeCast* design directly targets communication patterns that dominate autonomous IoT systems in both consumer and industrial settings, and is analyzed formally and evaluated for scalability in this paper.

A. Research Problem

Our goal is to design a secure and decentralized GKE protocol tailored to IoT communication patterns. The protocol must:

- **Eliminate central dependence:** Operate without, and no reliance on, a single trusted gateway.

- **Provide strong security:** Authenticate devices, ensure key confidentiality and freshness, and resist known attacks.
- **Handle dynamic membership:** Guarantee that new devices cannot access past communications and revoked devices cannot access future ones.
- **Remain lightweight:** Achieve low pairing time, computation, and storage costs, enabling deployment on constrained hardware.
- **Scale efficiently:** Support large, heterogeneous deployments with predictable overhead.

In addition to these core requirements, the protocol must also satisfy further properties that are summarized in Table I.

TABLE I: Protocol Requirements: Security Goals

Security Goal	**Description [7], [8]**
Perfect Forward Secrecy (PFS)	Ensures that compromise of long-term keys does not affect past session keys.
Post-Compromise Security	Even if a device is compromised at some point, future communications can remain secure after recovery.
Resistance to Known-Key Attack	Prevents past session key compromise from affecting future keys or allowing impersonation.
Key Confirmation	Confirms both parties share the same session key, preventing impersonation attacks.
Key Freshness	Ensures each session key is newly generated and not reused.
Key Control	Guarantees that both parties contribute to the key, preventing key-forcing by either party.
Mutual Entity Authentication	Both parties verify each other's identity in one-to-one communication.
Unilateral Entity Authentication	Receiver verifies sender's identity, suitable for one-to-many communication.

B. Our Contribution

In this work, we present *TreeCast*, a novel decentralized GKE protocol designed for the communication needs of IoT environments. Our main contributions are:

1) **Tree-based GKE.** We design a binary-tree structure where sibling nodes perform hashed Diffie–Hellman exchanges to recursively derive parent and root keys. This enables secure *unicast*, *multicast*, and *broadcast* communication in IoT networks.
2) **Dynamic Membership.** We introduce *localized rekeying*, allowing efficient device addition and revocation while preserving confidentiality of past and future messages.
3) **Security and Scalability.** We provide a formal security proof under standard hardness assumptions and analyze computational and storage costs, showing logarithmic growth with group size. This demonstrates the protocol's suitability for large, heterogeneous, and resource-constrained IoT deployments.

II. Related Work

To understand the current state of the art in secure key establishment for IoT systems, we have thoroughly reviewed several works in this domain. We highlight a few that are highly relevant to our work. These works cover both centralized and decentralized approaches, offering a broad perspective on existing techniques.

Few context-based pairing protocols, such as Perceptio [9], which introduces a novel approach that supports heterogeneous sensors, using event timings rather than raw sensor data. It requires sensor calibration for threshold tuning and assumes boundary security, restricting its use in public or shared spaces. Moreover, pairing time depends on activity frequency, with low-activity environments requiring artificial event injection at the cost of added complexity. IOTCUPID [10] uses Partitioned-GPAKE [9], assuming physical proximity and shared environmental context to establish cryptographic group keys without active user involvement. However, its revocation mechanism is inefficient, as adding or removing devices requires the reinitialization of the entire protocol, limiting scalability. In addition, it relies on a shared environmental context, making it unsuitable for open environments.

In HIBE (Hierarchical Identity-based Encryption) protocols such as the T-HIBE protocol [11], it splits the root PKG (Private Key Generator) among multiple non-colluding PKGs using Shamir secret sharing, which reduces the usual key-escrow problem. However, ciphertext size increases with hierarchy depth, and there is no immediate revocation, and updates at higher levels require all child nodes to reinitialize, which affects scalability. In addition, JEDI [12] built on WKD-IBE [13], enabling flexible and scalable key management. However, as in IBE, the key escrow problem and a single point of failure still persist, and the ciphertext size increases with the number of revoked devices, which affects the scalability.

In verifiable secret sharing schemes [14], the dealer's computational cost increases linearly with the number of participants, leading to scalability challenges for large IoT networks. In addition, the schemes are vulnerable to a single point of failure due to their dependence on the dealer.

III. TreeCast

We propose *TreeCast*, a key establishment protocol for secure IoT communication, built on a binary tree with Hashed Diffie–Hellman (HDH). TreeCast, inspired by the asynchronous ratcheting tree of Gordon et al. [14], incorporates unicast, multicast and broadcast communication and uses a blockchain-based PKI for decentralized identity management, public keys, and certificates, ensuring tamper-proof updates and verifiability, and includes a formal security proof, making it efficient and secure for resource-constrained IoT devices.

IoT devices are placed in leaf nodes of the binary tree. Each device derives its parent's secret via HDH from its sibling's public key, recursively continuing until the root key is obtained, which serves as the group key to broadcast any message. Intermediate node keys enable efficient multicast within subtrees. Public keys are stored on the blockchain by multiple CAs (Certificate Authorities), avoiding reliance on a single trusted party.

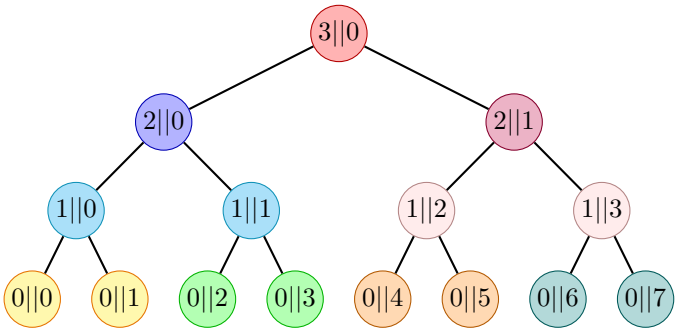

Fig. 1: Identities of the devices in the binary tree

To reduce overhead, public key methods are used only for session key establishment; subsequent communication uses lightweight symmetric keys (e.g., AES-128). The setup is one-time: node addition or revocation requires only local updates, not a full recomputation.

TreeCast is well-suited for large-scale open environments such as airports, railway stations, or industrial facilities, where the communication channel is exposed to potential adversaries. In such scenarios, when the same message needs to be sent to multiple devices, intermediate nodes can be used for efficient multicast communication.

A. Identity Assignment:

First, the CAs assign a nonnegative integer number as an identity for an IoT device. Subsequently, the devices assign IDs to their respective parent nodes independently of the CAs. The CAs will just approve it. With this identity assignment, we can revoke a device without affecting the IDs of the remaining nodes.

If the ID of a device is $0n$, then it assigns the ID of its parent node as:

$$\text{ParentID}(x_{0n}) = 1 \,||\, \left\lfloor \frac{n}{2} \right\rfloor,$$

So, in general, we have the following.

$$\text{ParentID}(x_{mn}) = (m+1) \,||\, \left\lfloor \frac{n}{2} \right\rfloor,$$

where, x_{ij} denotes the $(j+1)^{\text{th}}$ node in $(i+1)^{\text{th}}$ level from leaf/base level and $||$ denotes concatenation.

IV. Assumptions & Threat Model

A. Assumptions

Definition IV.1. Second Preimage Resistant:

Second preimage resistant is a property of a hash function $H : A \to B$ which holds when, for any given $m \in A$, it is "hard" to find an $m' \in A$, $m' \neq m$, such that

$$H(m) = H(m').$$

Definition IV.2. Hashed Decisional Diffie-Hellman (HDDH) Assumption: *[15]*

The Hashed Decisional Diffie-Hellman (HDDH) assumption states that no efficient PPT algorithm can distinguish between the following two distributions with non-negligible advantage:

- $(q, g, g^a, g^b, H(g^{ab}))$, *and*
- (q, g, g^a, g^b, r)

where $a, b, r \xleftarrow{R} \mathbb{Z}_q$.

Formally, For every PPT, 0/1-valued algorithm $\mathcal{A}$, the HDDH assumption is defined as:

$$\begin{aligned}\mathsf{Adv}_{\mathcal{A}}^{\mathsf{HDDH}}(\lambda) = \Big| &\Pr\left[\mathcal{A}(q, g, g^a, g^b, H(g^{ab})) = 1\right] \\ &- \Pr\left[\mathcal{A}(q, g, g^a, g^b, r) = 1\right]\Big| \\ &< \mathsf{negl}(\lambda).\end{aligned}$$

B. Threat Model:

CAs are assumed to be *honest-but-curious*. The devices will share the public keys to the CAs via a secure channel. Each device is equipped with a secure hardware chip (such as TPM [16]) that holds a secret attestation key, which the device manufacturer certifies. During initialization, the device proves that it owns this key using a challenge-response method, ensuring that only genuine devices get certificates. So, the trust also lies with the device manufacturer.

An adversary is assumed to have complete knowledge of the protocol and complete access to the communication channel. This includes the ability to intercept, replay, modify, or delete messages transmitted over the network. A malicious device may generate arbitrary secret keys and send them to the sibling nodes during key updates in an attempt to compromise the group key.

In addition, adversaries may attempt to impersonate legitimate devices and send incorrect or misleading messages to disrupt the system. This makes the protocol vulnerable to potential attacks such as eavesdropping, replay attack, man-in-the-middle attack, etc.

C. TreeCast Protocol

1) Initialization: The central authority announces the curve equation, the group generator, that is, the group parameters hash function, makes these parameters publicly available, and authenticates a set of certificate authorities (CAs). Each device i generates a Diffie–Hellman key pair $(g^{k_{0i}}, k_{0i})$ and a signing/verification pair $(\texttt{Sign}_{x_{0i}}, \texttt{Verf}_{x_{0i}})$, then sends $g^{k_{0i}}$ and $\texttt{Verf}_{x_{0i}}$ over a secure channel to the CAs, which record them in a blockchain-based PKI (and devices keep their private keys. Multiple CAs eliminate a single point of failure and decentralize trust. The CAs assign environment-specific unique IDs, publish the associated public keys and (e.g., ECDSA-signed) certificates, and devices retrieve public keys of other devices directly from the blockchain. Devices then deterministically construct IDs for their parent and intermediate nodes; the CAs can verify group membership/authorization, and these IDs are then mapped to the leaf nodes of a binary tree, based on a required IoT environment.

2) Setup:

- Each device x first fetches the public key of $s(x)$ ($s(x)$ denotes the sibling node of x) from the blockchain with the desired ID by calling the smart contract. If it has ID

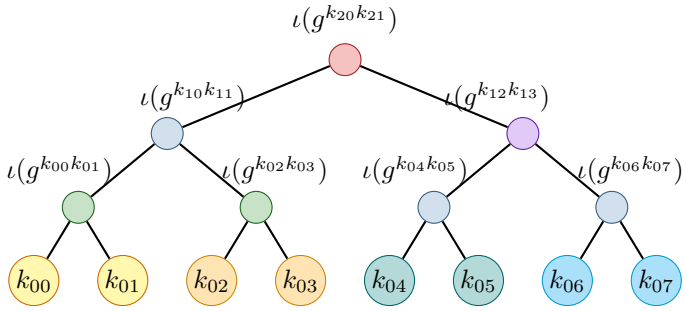

Fig. 2: Recursive key derivation in a binary tree structure (colour-enhanced for clarity)

$0||n$ where n is even, then it will ask for the public key of the devices with identity $0||(n+1)$ and $0||(n-1)$ otherwise.

- Each device x computes the secret key for $p(x)$ ($p(x)$ denotes the parent node of x) using the public key of $s(x)$ in the tree and its own secret key. Let a node with identity $m||n$ be denoted as x_{mn} as shown in Figure 1 and k_{ij} denote the secret key of x_{ij}. Let k_1 be the secret key of x_{0n}, and let g^{k_2} be the public key of its sibling node $x_{0(n+1)}$, if n is even, $x_{0(n-1)}$ otherwise.
- The secret key of the parent node $x_{1(\lfloor n/2 \rfloor)}$ is computed as:

$$sk_{\text{parent}} = \iota\left((g^{k_2})^{k_1}\right) = \iota\left(g^{k_1 k_2}\right),$$

where $\iota(\cdot)$ is a hash function from $\mathbb{G}$ to $\mathbb{Z}_q$, where q is a prime and $|\mathbb{G}| = q$.

- The corresponding public key of the parent node becomes:

$$pk_{\text{parent}} = g^{sk_{\text{parent}}} = g^{\iota(g^{k_1 k_2})}.$$

- Importantly, if a device x has no sibling node, then it selects $k \xleftarrow{R} \mathbb{Z}_q$, where $\xleftarrow{R}$ denotes uniform random sampling, and set it as secret key for its own and the secret key for $p(x)$, thus:

$$sk_{\text{parent}} = k$$

- Each device computes its public key, submits it to the CAs for verification, whether the device is a leaf node of that subtree, rooted at that abstract node, and after approval, the key is uploaded to the blockchain with the identity tag $x_{1(\lfloor n/2 \rfloor)}$.
- The device then recursively retrieves the sibling public keys of its parent node from the blockchain, derives higher-level keys (e.g., $x_{2(\lfloor \lfloor n/2 \rfloor /2 \rfloor)}$), and repeats the process until the root key is obtained (Fig. 2).

3) Session Key Establishment: Public-key operations are relatively expensive on resource-constrained devices, so we employ a hybrid encryption approach: a long-term public key exchange is used only to establish a short-term session key, and thereafter we will use a symmetric cipher (e.g., AES-128) to encrypt the actual messages.

- **Without Perfect Forward Secrecy (PFS):**
  If PFS is not required in our IoT environment, then we can use the following session key establishment method. Let A be a sender device with a long-term secret key α (so its public key is g^{α}), and let B be a receiver, either a single device, a subset of the group, or the entire group with the long-term secret key β (public key g^{β}) and gk as the root key. We derive a session key at round $i \in \mathbb{Z}_{\geq 0}$ by

$$K^{i}_{\text{session}} = \text{KDF}\left(g^{\alpha\beta} \,\|\, i\right).$$

  or for broadcasting,

$$K^{i}_{\text{session}} = \text{KDF}\left(gk \,\|\, i\right).$$

  If an adversary compromises A or B and learns α or β, then they can immediately recompute $g^{\alpha\beta}$ and hence recover all the keys of the previous session.
- **Unicast communication:**
  In unicast communication, one device will retrieve the long-term public key of the receiver's device and use it to generate the session key using fresh ephemeral keys generated by both devices.
  - Let a and b be the long-term secret keys of parties A and B, respectively (with public keys g^a and g^b).
  - In each new session, A samples a fresh ephemeral exponent x, computes, and sends g^x to B after signing on it.
  - Likewise, B samples a fresh ephemeral exponent y, computes, and sends g^y to A after signing on it as well.
  - Then both parties derive the shared session key as

$$K_{\text{session}} = H\left(g^{ab} \,\|\, g^{xb} \,\|\, g^{ay} \,\|\, g^{xy}\right).$$

- **Multicast communication:**
  In multicast communication, one device will retrieve the long-term public key of the least-common-ancestor node of all the targeted nodes and use it to generate the session key using a ephemeral key of the sender node itself. Moreover, in broadcast communication, the root key will be used to generate the session key. The receiver nodes will only verify the signature on the ephemeral key.
  - Let a and b be the long-term secret keys of parties A and B where A is a single device and B is a subset of the entire group or the entire group, respectively (with public keys g^a and g^b).
  - In each new session, A samples a fresh ephemeral exponent x, computes, and sends g^x to B, after signing on it.
  - Then both parties derive the shared session key as

$$K_{\text{session}} = H\left(g^{ab} \,\|\, g^{xb}\right).$$

4) Key Update:

Step 1: Update Trigger: A periodic or event-driven key update is initiated for leaf node x_{06} in Figure 3.

Step 2: Leaf Key Regeneration: Node x_{06} generates a fresh pair of private–public keys $\left(k'_{06}, g^{k'_{06}}\right)$.

Step 3: Path Rekeying: Node x_{06} recomputes all intermediate secret keys and corresponding public keys along the $path(x_{06})$ where $path(x)$ ancestor path from node x to the root node, and they are sent to the CAs, which upload them to the blockchain.

Step 4: Distribution of New Keys: Node x_{06} encrypts each new secret key under the public key of the nodes in $copath(x_{06})$, where $copath(x)$ denotes a set of siblings of nodes in $path(x)$ excluding the nodes in $path(x)$, and multicasts these secret keys to existing devices.

Step 5: Verification: All recipients retrieve the updated public keys from the blockchain, decrypt their shares, and verify the consistency of the rekey operation.

5) Revocation:

Step 1: Revocation Trigger: A device is revoked from the system, here x_{37} as shown in Figure 3. (e.g., due to compromise).

Step 2: Certificate Revocation: The CAs revoke the certificate of the revoked device x_{07} and update the list on the blockchain.

Step 3: Sibling Key Update: The sibling node x_{06} of the revoked leaf node performs the Key Update method described above.

6) Addition: The device addition procedure is similar to the revocation method.

D. Computational and Space Cost

Let n be the total number of devices and $h = \lceil \log_2 n \rceil$ the height of the binary tree (assuming $n = 2^m$ for simplicity).

Per-Device Computational Cost: Each device computes its own public key and derives the parent keys up to the root. The asymptotic operation counts are summarized below:

Operation	Per Device
Exponentiations	$1 + 2h = 1 + 2\lceil \log_2 n \rceil$
Hash evaluations	$h = \lceil \log_2 n \rceil$

Root Computation Time: Since all devices at a given level operate in parallel, the total time to compute the root key is dominated by the tree height:

$$T_{\text{root}} = hT_\iota + (2h+1)T_e$$

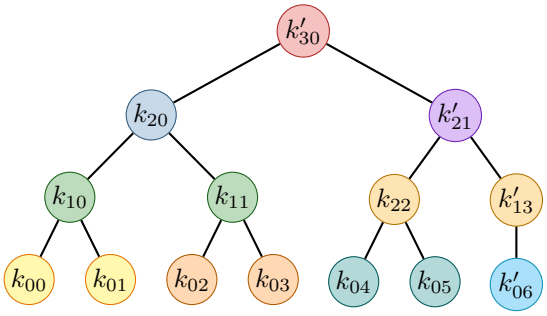

Fig. 3: Updated Tree after Revocation of x_{07} and Re-Keying by x_{06} (I)

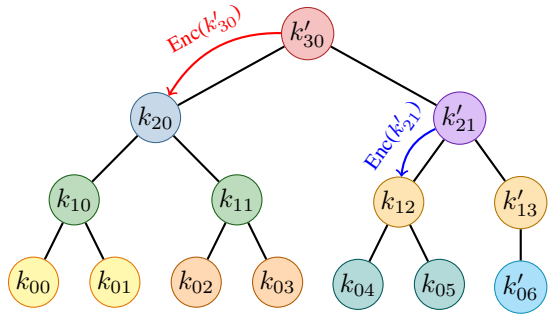

Fig. 4: Updated Tree after Revocation of x_{07} and Re-Keying by x_{06} (II)

where T_ι and T_e are the costs of a hash evaluation and an exponentiation, respectively.

Space Requirement: Each device stores secret keys along its path to the root, i.e., $\log_2 n$ keys. Assuming 128-bit keys:

$$\text{Storage per device} = 128 \log_2 n \text{ bits} \approx 0.128 \log_2 n \text{ KB}.$$

For example, a Wio Terminal [17] with 192 KB SRAM can easily support this storage overhead.

E. Remarks:

- To add an extra layer of security towards PFS, in a closed environment, such as a smart home, where PFS is necessary, we can incorporate a context-based nonce collected from the environment, similar to the approach used in IOTCUPID [10]. This nonce can then be used in the generation of session keys. Although this does not provide PFS, it offers a security enhancement towards achieving PFS.
- We extend our protocol to nested IoT environments (e.g., smart buildings or smart cities) by organizing devices hierarchically in a binary tree (Fig. 5). Devices sharing a physical location (e.g., the same room or floor) derive a common secret, enabling efficient key management across all layers. By assigning the proper IDs to each device, we can realize this structure, which also applies to multi-domain scenarios such as industrial IoT systems.

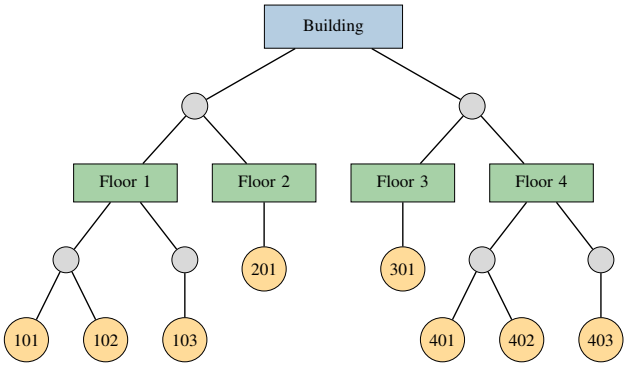

Fig. 5: Illustration of generalization of the Protocol in a nested environment

TABLE II: Comparative Analysis of Protocols Across Key Security Properties

Protocol	P_1	P_2	P_3	P_4
JEDI [12]	✓	✓	✗	✓
IOTCUPID [10]	✗	✗	✓	✓
Perceptio [1]	✗	✗	✗	✓
T-HIBE [11]	✓	✗	✓	✓
Symmetric Key [18]	✓	✗	✓	✗
VSS [14]	✗	✗	✗	✗
This paper [TreeCast]	✓	✓	✓	✓

Protocol	P_5	P_6	P_7
JEDI [12]	✙	✗	✗
IOTCUPID [10]	-	-	✓
Perceptio [1]	-	✓	✓
T-HIBE [11]	✗	✓	✓
Symmetric Key [18]	✗	✓	✓
VSS [14]	-	✓	✓
This paper [TreeCast]	◐	✓	✓

Property Assignments (Left Table):

P_1 = Scalability,
P_2 = Efficient Immediate Revocation,
P_3 = Avoid single TTP,
P_4 = Decentralized.

Property Assignments (Right Table):

P_5 = Perfect Forward Secrecy (PFS),
P_6 = Post-Compromise Security,
P_7 = Efficiency for Constrained Devices,

Symbols for Protocol Properties: ✓ Fully supports ✗ Does not support ◐Partially supports ✙ Can be extended

- We cannot use multilinear maps top generalize our protocol since, Existing multilinear maps such as GGH13 [19] and CLT13 [20] require intensive operations of the order of hundreds of milliseconds to seconds per evaluation on moderately powerful hardware. This makes them impractical for resource-constrained IoT devices. We also have some statistical attacks on existing multilinear maps [21], [22]. Otherwise, we could develop asymmetric n-ary key agreement trees leveraging multilinear maps.

V. Evaluation

The security and efficiency of any cryptographic protocol must be rigorously evaluated to justify its practical applicability, especially in resource-constrained environments. This section presents a complete assessment of the TreeCast protocol, focusing on its computational cost and communication efficiency.

A. Scalability

From the root key computation formula, it is evident that TreeCast achieves high efficiency even in large-scale IoT deployments. For instance, in a network with 10^6 devices, each device requires only 20 hash evaluations and 41 modular exponentiations. This shows that the protocol remains highly scalable even in massive IoT environments.

B. Comparison with other works

As outlined in Section II, several group key establishment protocols have been proposed in the literature, ranging from symmetric-key solutions to identity-based and context-driven mechanisms. Although these approaches address subsets of the problem space, they fall short when evaluated against the combined requirements of scalability, dynamic membership, decentralization, and efficiency on constrained devices.

Table II summarizes the comparison between representative protocols and our TreeCast protocol across eight key security and performance properties. Protocols such as JEDI [12] and T-HIBE [11] achieve scalability but impose high computational overhead, which limits their deployment on lightweight IoT devices. Context-based pairing mechanisms like Perceptio [23] and IOTCUPID [10] remove reliance on centralized infrastructure but depend heavily on environmental cues and user interaction, making them unsuitable for large-scale or unattended IoT deployments. Symmetric approaches relying on a key distribution center are lightweight in isolation [18] but incur high communication overhead whenever devices join or leave, which affects scalability.

In contrast, TreeCast provides a balanced design. It supports *scalability* by organizing devices in a binary tree and localizing rekey operations. It ensures *efficient immediate revocation* by updating only along affected paths, thereby avoiding system-wide resets. It avoids reliance on a single trusted third party, since trust is distributed across certification authorities and reinforced by a blockchain-based PKI. Furthermore, it achieves strong security properties, including authentication, key confirmation, post-compromise security, and partial forward secrecy, while remaining efficient for constrained devices. This combination of properties distinguishes our work from prior approaches and demonstrates its suitability for securing large-scale, heterogeneous IoT deployments.

VI. Security Analysis

In this section, we present a comprehensive analysis of the TreeCast protocol, focusing on its resilience against both passive adversaries and a range of active attacks, including man-in-the-middle (MitM), replay, and known-key attacks.

A. Security Against Passive Adversaries

We first consider a passive adversary who can eavesdrop on communication but does not alter messages. We show that our protocol preserves confidentiality under this threat model.

Let $\mathbb{G}$ be a cyclic group of prime order q with generator g. Let $H : \mathbb{G} \to \mathbb{Z}_q$ be a hash function. We define the following security game to capture the adversary's advantage:

XY Game

An adversary $\mathcal{A}$ is given a tuple

$$(g,\, g^x,\, g^y,\, g^{H(g^{xy})})$$

for random $x, y \xleftarrow{R} \mathbb{Z}_q^*$, and must output a guess $T' \in \mathbb{Z}_q$ for the value $H(g^{xy})$.

XY Advantage:

$$\mathrm{Adv}_{\mathrm{XY}}(\mathcal{A}) = \Pr[\mathcal{A}(g, g^x, g^y, g^{H(g^{xy})}) = H(g^{xy})].$$

Lemma: Hardness of XY

If the HDDH assumption holds in $\mathbb{G}$, then no PPT (Probabilistic Polynomial Time) adversary can solve the XY problem with non-negligible advantage. Equivalently,

$$\mathrm{Adv}_{\mathrm{XY}}(\mathcal{A}) \;\leq\; \mathsf{negl}(\lambda).$$

Proof Sketch. Assume a PPT adversary $\mathcal{A}$ exists with

$$\mathrm{Adv}_{\mathrm{XY}}(\mathcal{A}) \;\geq\; \tfrac{1}{q} + \mathsf{negl}(\lambda).$$

We construct a distinguisher $\mathcal{D}$ for the HDDH problem:

1) Receive (g, g^x, g^y, T) from challenger.
2) Compute g^T and forward (g, g^x, g^y, g^T) to $\mathcal{A}$.
3) If $\mathcal{A}$ outputs $T' = T$, then $\mathcal{D}$ outputs 1, else 0.

If $T = H(g^{xy})$, the view is identical to XY game, so $\Pr[\mathcal{D} = 1] = \mathrm{Adv}_{\mathrm{XY}}(\mathcal{A})$. If T is random, the success probability is exactly $1/q$. Thus,

$$\mathrm{Adv}_{\mathrm{HDDH}}(\mathcal{D}) \;=\; \left|\mathrm{Adv}_{\mathrm{XY}}(\mathcal{A}) - \tfrac{1}{q}\right| \;\geq\; \mathsf{negl}(\lambda),$$

contradicting the HDDH assumption. □

Theorem: Hardness of Computing Root Key

Let $\mathcal{B}_h$ be the set of all perfect binary trees of height h. Given $B \xleftarrow{R} \mathcal{B}_h$, $K \xleftarrow{R} (\mathbb{Z}_q^*)^n$ with $n \leq 2^h$, define the public view

$$\mathsf{view}(h, K, B) := \{g^{k_{ij}} \mid \forall i, j\},$$

and root key $\mathrm{gk} = x_{h0} = \iota(g^{x_{(h-1)0}x_{(h-1)1}})$.

Claim: Under the HDDH assumption, no PPT adversary can compute gk given $\mathsf{view}(h, K, B)$.
Proof idea. The structure of the proof is similar to that of Barua et al. [24]
- **Base case:** By Lemma above, XY is hard, hence so is $h = 1$.
- **Induction:** Assume true for height $h - 1$.
- **Step:** If an adversary computes gk at height h, then it either (a) breaks HDDH, or (b) recovers children keys at level $h - 1$. Both contradict the assumptions. □

B. Security Against Active Adversaries

We now analyze the protocol in the presence of active adversaries, who are capable of modifying, injecting, or replaying messages in addition to passive eavesdropping.

- **Forward Secrecy:** TreeCast provides different levels of forward secrecy depending on the communication mode. In the case of *multicast*, perfect forward secrecy is not achieved. Instead, the protocol attains *partial forward secrecy*: if the long-term secret key of the sender is compromised, past session keys remain secure since each session incorporates a fresh ephemeral key x that is securely erased after use. However, if the long-term secret key of a receiver is compromised, an adversary can recover all past session keys for that receiver.
  In contrast, *unicast* communication achieves perfect forward secrecy. Each session derives its key from fresh ephemeral values x and y, ensuring that even if the long-term secrets of one or both parties are later compromised, previously established session keys cannot be reconstructed.
- **Limitations of Perfect Forward Secrecy in Multicast:** In multicast communication, perfect forward secrecy would require an ephemeral–ephemeral exchange between the sender A and every member of the group B. This is equivalent to generating a fresh *group ephemeral key* for each session that is completely independent of the long-term secrets. Such a mechanism introduces significant computational and communication overhead, which is impractical for constrained IoT devices. Therefore, our design provides only *partial* forward secrecy in the multicast setting.
- **Post-Compromise Security:** If the long-term secret key of a device is compromised, an adversary can derive all keys along the corresponding $path(x)$, where x denotes the compromised device. To recover, the compromised device generates a fresh secret key, recomputes all keys along $path(x)$, and updates the Certification Authorities (CAs) as well as the other devices in the domain. This rekeying process restores security for subsequent sessions. In the case of *unicast* communication, the use of ephemeral secrets further strengthens post-compromise security by ensuring that future session keys remain protected, even if the long-term key has been exposed.
- **Man-in-the-Middle (MitM) Attack:** All public keys are certified by trusted CAs and immutably stored on the blockchain. Devices verify CA signatures before deriving parent-node keys, ensuring that any modification or injection of false information is detected. In addition, ephemeral keys are signed and verified, which prevents an adversary from successfully mounting MitM attacks.
- **Replay Attack and Key Freshness:** Each session key is derived through a key derivation function (KDF) that incorporates a nonce or an ephemeral secret. Since keys are never reused, any replayed message fails verification and is discarded. This mechanism not only provides resistance against replay attacks but also guarantees key freshness for every session.
- **Key Confirmation:** All devices can verify that they hold the same session key. This is achieved either through counter synchronization in the absence of forward secrecy

or by combining ephemeral and long-term keys in the partial forward secrecy setting.

- **Key Control:** Every device contributes its long-term secret during key generation, ensuring that no single device can determine the root key or any intermediate node key. Furthermore, session keys depend on counters (in the no-PFS setting) or ephemeral secrets (in the partial-PFS setting), preventing manipulation by any single party.
- **Known-Key Attack:** Session keys are derived using a key derivation function or a hash function assumed to be second-preimage resistant (and therefore preimage resistant). This prevents an adversary who has learned one session key from recovering the corresponding inputs or predicting future session keys.
- **Authentication:** Ephemeral keys are signed with long-term signature keys and verified through the blockchain-based PKI. This provides mutual authentication in *unicast* communication and unilateral authentication in *multicast* and *broadcast*.

VII. Conclusion

This work introduced *TreeCast*, a decentralized group key establishment protocol tailored for the unique requirements of resource-constrained IoT environments. By organizing devices in a binary tree and applying hashed Diffie–Hellman key exchange, TreeCast achieves efficient and scalable key management across the three fundamental communication modes: *unicast*, *multicast*, and *broadcast*. The scheme ensures that secure one to one, one to many, and one to all communication can be realized without reliance on a central gateway, directly addressing the dominant communication patterns found in both consumer and industrial IoT settings. Examples include secure data sharing and alert dissemination in smart homes, as well as coordinated control and emergency signaling in industrial systems.

TreeCast was shown to provide strong security guarantees under standard hardness assumptions. It ensures authentication, session key confidentiality, and key confirmation, while also supporting post-compromise security and partial forward secrecy. Dynamic membership is efficiently supported through localized rekeying, enabling practical handling of device addition and revocation without system-wide resets. Furthermore, the use of a blockchain-based PKI enhances trust and verifiability by eliminating single points of failure. Together, these properties demonstrate that TreeCast offers a scalable, secure, and lightweight framework for key management in large-scale, heterogeneous IoT deployments, making it a practical solution for securing autonomous device-to-device communication.

Acknowledgement

This work was supported in part by CyberSecurity Research Flanders with reference number VR20192203, and by the European Union's Horizon Research and Innovation program under grant agreement No. 101119747 (TELEMETRY).

References

[1] T. Alam, "A Reliable Communication Framework and Its Use in Internet of Things (IoT)," *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, 2018.

[2] S. Duttagupta, A. Kolozyan, G. Nicolas, B. Preneel, and D. Singelee, "What's the Matter? An In-Depth Security Analysis of the Matter Protocol," Cryptology ePrint Archive, Paper 2025/1268, 2025. [Online]. Available: https://eprint.iacr.org/2025/1268

[3] J. McCune, A. Perrig, and M. Reiter, "Seeing-is-believing: Using Camera Phones for Human-Verifiable Authentication," in *IEEE Symposium on Security and Privacy*, 2005.

[4] T. Zhang, X. Yi, R. Wang, Y. Wang, C. Yu, Y. Lu, and Y. Shi, "Tap-to-Pair: Associating Wireless Devices with Synchronous Tapping," *ACM Interactive, Mobile, Wearable and Ubiquitous Technologies*, 2018.

[5] X. Li, Q. Zeng, L. Luo, and T. Luo, "T2Pair: Secure and Usable Pairing for Heterogeneous IoT Devices," in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2020.

[6] X. Luo, L. Yin, C. Li, C. Wang, F. Fang, C. Zhu, and Z. Tian, "A Lightweight Privacy-Preserving Communication Protocol for Heterogeneous IoT Environment," *IEEE Access*, 2020.

[7] D. Boneh and V. Shoup, *A Graduate Course in Applied Cryptography*, 2017. [Online]. Available: https://toc.cryptobook.us/book.pdf

[8] P. C. v. O. Alfred J. Menezes and S. A. Vanstone, *Handbook of Applied Cryptography*. CRC Press, 1996.

[9] D. Fiore, M. I. G. Vasco, and C. Soriente, "Partitioned Group Password-Based Authenticated Key Exchange," 2017, cryptology ePrint Archive, Paper 2017/141. [Online]. Available: https://eprint.iacr.org/2017/141.pdf

[10] H. Farrukh, M. O. Ozmen, F. Kerem Ors, and Z. B. Celik, "One key to rule them all: Secure group pairing for heterogeneous IoT devices," in *2023 IEEE Symposium on Security and Privacy (SP)*, 2023.

[11] S. Duttagupta, D. Singelée, and B. Preneel, "T-HIBE: A Novel Key Establishment Solution for Decentralized, Multi-Tenant IoT Systems," in *2022 19 IEEE Annual Consumer Communications & Networking Conference (CCNC)*, 2022.

[12] S. Kumar, Y. Hu, M. P. Andersen, R. A. Popa, and D. E. Culler, "JEDI: Many-to-Many End-to-End Encryption and Key Delegation for IoT," in *28th USENIX Security Symposium*, 2019.

[13] M. Abdalla *et al.*, "Generalized Key Delegation for Hierarchical Identity-Based Encryption," 2007, cryptology ePrint Archive, Paper 2007/221. [Online]. Available: https://eprint.iacr.org/2007/221.pdf

[14] L. Lu and J. Lu, "A lightweight verifiable secret sharing scheme in IoTs," Cryptology ePrint Archive, Paper 2022/395, 2022. [Online]. Available: https://eprint.iacr.org/2022/395

[15] M. Abdalla, M. Bellare, and P. Rogaway, "DHAES: An encryption scheme based on the Diffie-Hellman problem," Cryptology ePrint Archive, Paper 1999/007, 1999. [Online]. Available: https://eprint.iacr.org/1999/007

[16] "Tpm attestation in azure iot dps," https://learn.microsoft.com/en-us/azure/iot-dps/concepts-tpm-attestation.

[17] "Get started with wio terminal," https://wiki.seeedstudio.com/Wio-Terminal-Getting-Started/.

[18] X. Luo *et al.*, "A Lightweight Privacy-Preserving Communication Protocol for Heterogeneous IoT Environment," *IEEE Access*, vol. 8, 2020.

[19] S. Garg, C. Gentry, and S. Halevi, "Candidate Multilinear Maps from Ideal Lattices," in *Advances in Cryptology – EUROCRYPT 2013*, T. Johansson and P. Q. Nguyen, Eds., 2013.

[20] J.-S. Coron, T. Lepoint, and M. Tibouchi, "Practical multilinear maps over the integers," Cryptology ePrint Archive, Paper 2013/183, 2013. [Online]. Available: https://eprint.iacr.org/2013/183

[21] J. H. Cheon, C. Lee, and H. Ryu, "Cryptanalysis of the New CLT Multilinear Maps," Cryptology ePrint Archive, Paper 2015/934, 2015. [Online]. Available: https://eprint.iacr.org/2015/934

[22] L. Ducas and A. Pellet-Mary, "On the Statistical Leak of the GGH13 Multilinear Map and some Variants," Cryptology ePrint Archive, Paper 2017/482, 2017. [Online]. Available: https://eprint.iacr.org/2017/482

[23] J. Han, A. J. Chung, M. K. Sinha, M. Harishankar, S. Pan, H. Y. Noh, P. Zhang, and P. Tague, "Do You Feel What I Hear? Enabling Autonomous IoT Device Pairing Using Different Sensor Types," in *IEEE Symposium on Security and Privacy (S&P)*, 2018.

[24] R. Barua, R. Dutta, and P. Sarkar, "Extending Joux's Protocol to Multi Party Key Agreement," Cryptology ePrint Archive, Paper 2003/062, 2003. [Online]. Available: https://eprint.iacr.org/2003/062